# ADAPTIVE RESOURCE QUARANTINE

*Problem definition, literature and gaps in existing systems*

**REVIEW I**

Rithika.S (24BCE2984)

K. Vaishnavi (24BCT0183)

Vellore Institute of Technology

Operating Systems Lab | BCSE303P

Slot: L15 + L16

Faculty: Prof. Anisha M. Lal

Combined review record and technical disclosure
30 October 2026

Project repository

## 1. Abstract

Operating systems share CPU time, memory and input/output resources among running processes. Sustained resource pressure can make other applications less responsive, but high utilization alone does not show that a process is faulty. Adaptive Resource Quarantine (ARQ) explores a graduated response between passive observation and termination. A Python monitor collects process telemetry, a rule engine combines resource signals and EWMA tracking, and a controller applies staged restrictions through scheduling priority and signal-based CPU duty cycles. The original recovery path reduces restrictions after repeated low scores. This combined report retains that project direction and extends its evaluation to a specific limitation: throttling can itself create the low readings used to infer recovery. An opt-in research module therefore tests a temporary allocation increase between two restricted control windows. It checks protected-service latency, measurement stability and CPU consumption relative to the larger grant before approving one relaxation. Functional tests and constructed scenarios support the implemented logic; they do not establish real-world detection accuracy, production safety or patentability.

## 2. Introduction

ARQ studies a practical operating-systems question: how should a controller respond when a process consumes resources unusually? A brief compiler burst can be legitimate, while sustained excessive demand may disrupt neighboring work. A response should therefore be proportional to the evidence and explicit about its limits.

This report separates three stages. Review I establishes the problem and related work. Review II describes the architecture, implementation and recorded outputs. Review III evaluates the available evidence, compares the mechanisms with existing approaches and identifies future work. A separate IDF-B section records the candidate research mechanism for professional review.

## 3. Problem Statement

Combine process telemetry and interpretable scoring with graduated execution restrictions and a justified method for relaxing those restrictions. The controller must distinguish a workload that has genuinely reduced demand from one whose observed usage is low only because the controller is restricting it.

## 4. Objectives

Collect CPU, resident/virtual memory, growth, I/O and thread observations where the host exposes them.

Use the existing resource scoring path to select monitoring and quarantine states.

Apply supported priority and signal-based duty controls, with restoration and cleanup behavior visible to the operator.

Evaluate Isolation Forest separately in shadow mode without treating its scores as calibrated probabilities.

Test a bracketed release-verification policy before a proposed CPU-duty relaxation.

Retain test logs, modeled scenarios and reported historical outputs with clear evidence boundaries.

## 5. Literature Survey

The following twenty scholarly works cover anomaly detection, process response, interference assessment and resource management. They are a technical comparison rather than an assertion that earlier systems lack recovery or adaptive control. The studies have not been run head-to-head against ARQ. References are listed after Review III.

### 5.1 Literature Comparison

| Study | Main contribution | Relevance and boundary |
|---|---|---|
| [1] Ibidunmoye, Hernandez-Rodriguez and Elmroth (2015). Performance Anomaly Detection and Bottleneck Identification. ACM Computing Surveys. | Surveys performance anomaly detection and bottleneck analysis. | Supports telemetry analysis; does not validate ARQ thresholds. |
| [2] Liu, Ting and Zhou (2008). Isolation Forest. ICDM. | Uses isolation trees to score unusual observations. | ARQ uses a standard optional model in shadow mode. |
| [3] Hochreiter and Schmidhuber (1997). Long Short-Term Memory. Neural Computation. | Introduces recurrent modeling of temporal dependencies. | Background only; no trained LSTM is established here. |
| [4] Malhotra et al. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. | Detects unusual sequences through reconstruction error. | Sequence reconstruction is prior work, not a new ARQ mechanism. |
| [5] Somayaji (2002). Operating System Stability and Security through Process Homeostasis. Doctoral thesis. | Explores behavioral process monitoring and adaptive response. | A foundational overlap with observe-and-respond systems. |

### 5.2 Literature Comparison

| Study | Main contribution | Relevance and boundary |
|---|---|---|
| [6] Lo et al. (2016). Improving Resource Efficiency at Scale with Heracles. ACM TOCS. | Manages colocated resources with feedback to protect latency. | Latency-driven allocation and recovery are established. |
| [7] Fried et al. (2020). Caladan: Mitigating Interference at Microsecond Timescales. OSDI. | Uses fast allocation and contention signals for isolation. | ARQ does not reproduce its scheduler or timing scale. |
| [8] Chen, Delimitrou and Martinez (2019). PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. ASPLOS. | Dynamically partitions resources for colocated interactive services. | Multi-service QoS feedback is not novel to this project. |
| [9] Ousterhout et al. (2019). Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. NSDI. | Adjusts CPU allocations to changing service demands. | Provides a scheduling comparison, not an ARQ implementation. |
| [10] Zhang et al. (2013). CPI2: CPU performance isolation for shared compute clusters. EuroSys. | Uses CPI observations to identify interference and throttle likely offenders. | Interference-aware selective throttling already exists. |

### 5.3 Literature Comparison

| Study | Main contribution | Relevance and boundary |
|---|---|---|
| [11] Delimitrou and Kozyrakis (2014). Quasar: Resource-Efficient and QoS-Aware Cluster Management. ASPLOS. | Combines classification with resource allocation and assignment. | Learned resource decisions are established research. |
| [12] Delimitrou and Kozyrakis (2013). Paragon: QoS-aware scheduling for heterogeneous datacenters. ASPLOS. | Accounts for hardware heterogeneity and workload interference. | A broader placement problem than local CPU-quarantine release. |
| [13] Mars et al. (2011). Bubble-Up: Increasing Utilization In Modern Warehouse Scale Computers Via Sensible Co-Locations. MICRO. | Applies tunable memory pressure to characterize interference sensitivity. | Active perturbation for interference assessment is known. |
| [14] Banga, Druschel and Mogul (1999). Resource Containers: A New Facility for Resource Management in Server Systems. OSDI. | Separates resource accounting principals from protection domains. | Resource management and security isolation are distinct. |
| [15] Hindman et al. (2011). Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. NSDI. | Coordinates fine-grained sharing across cluster frameworks. | Cluster allocation differs from this local recovery experiment. |

### 5.4 Literature Comparison

| Study | Main contribution | Relevance and boundary |
|---|---|---|
| [16] Verma et al. (2015). Large-scale cluster management at Google with Borg. EuroSys. | Describes large-scale workload placement and cluster management. | Deployment scale and guarantees are not comparable to ARQ. |
| [17] Schwarzkopf et al. (2013). Omega: flexible, scalable schedulers for large compute clusters. EuroSys. | Uses shared state and optimistic concurrency for scheduling. | Does not establish this prototype as a cluster scheduler. |
| [18] Ghodsi et al. (2011). Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. NSDI. | Defines multi-resource fair allocation. | ARQ does not implement DRF or prove allocation fairness. |
| [19] Rzadca et al. (2020). Autopilot: Workload Autoscaling at Google Scale. EuroSys. | Automates resource sizing and scaling for workloads. | Autoscaling evidence cannot be transferred to ARQ. |
| [20] Kulkarni et al. (2019). Pliant: Leveraging Approximation to Improve Datacenter Resource Efficiency. HPCA. | Uses approximation to reduce colocation contention. | ARQ changes CPU duty, not application approximation quality. |

## 6. Gaps and Project Scope

### 6.1 Fixed thresholds and changing workload phases

A fixed resource threshold is simple to interpret but may treat legitimate bursts as anomalies. ARQ adds burst handling and EWMA tracking, although several component scores still use configured fixed thresholds. Adaptive tracking should not be described as complete workload understanding.

### 6.2 Detection and response

Detection alone does not control resource use. ARQ connects observations to concrete actions, but signal-based throttling is execution control, not a security sandbox or complete memory/I/O isolation.

### 6.3 Restriction-masked recovery

The controller can observe low CPU usage because it reduced runnable time. Repeated low observations therefore need additional evidence before they justify restoration. This is the specific failure mode addressed by the research extension.

### 6.4 Existing feedback systems

Heracles, CPI2, PARTIES and related systems already manage interference and protect workloads [6-10]. The candidate contribution must be assessed as an exact method, not as the first adaptive quarantine or latency-aware controller.

### 6.5 Repeatable evaluation

An interface demonstration is not a detection-accuracy study. Labelled workloads, repeated trials, trustworthy CPU accounting and application-state checks are required before stronger performance or preservation claims are made.

# ADAPTIVE RESOURCE QUARANTINE

*Proposed methodology, architecture and implementation outputs*

**REVIEW II**

Rithika.S (24BCE2984)

K. Vaishnavi (24BCT0183)

Vellore Institute of Technology

Operating Systems Lab | BCSE303P

Slot: L15 + L16

Faculty: Prof. Anisha M. Lal

Combined review record and technical disclosure
30 October 2026

Project repository

## 7. Proposed Methodology

The base monitoring cycle reads process telemetry, updates the profile, computes a rule score, chooses a stage, applies its associated action and observes the next cycle. Isolation Forest is evaluated alongside that path. Its output is displayed separately and does not select the quarantine stage.

### 7.1 Resource signals

| Signal | Use and limitation |
|---|---|
| CPU utilization | Sigmoid score around a configured threshold; usage is affected by the current restriction. |
| Memory growth / RSS | Growth and absolute footprint contribute to a bounded score; CPU throttling is not a hard RSS limit. |
| I/O throughput | Read/write rate contributes where the operating system exposes it. |
| Burst deviation | Difference between CPU observation and its EWMA contributes to burst scoring. |
| Thread / virtual memory | Additional features for the optional Isolation Forest pipeline. |

### 7.2 Rule path

The default weighted score uses CPU, memory, I/O and burst weights of 0.40, 0.35, 0.15 and 0.10. Source code also applies a dominant-vector boost and early-cycle suppression. The CPU sigmoid denominator is 10 in the inspected source; the EWMA factor is 0.25. These constants require workload-specific evaluation.

### 7.3 State and configuration

Controller defaults use soft/hard thresholds of 0.65/0.85, while the existing demo and benchmark use 0.55/0.75. The report keeps these configurations distinct. The original restoration default requires four low-score cycles.

## 8. System Architecture

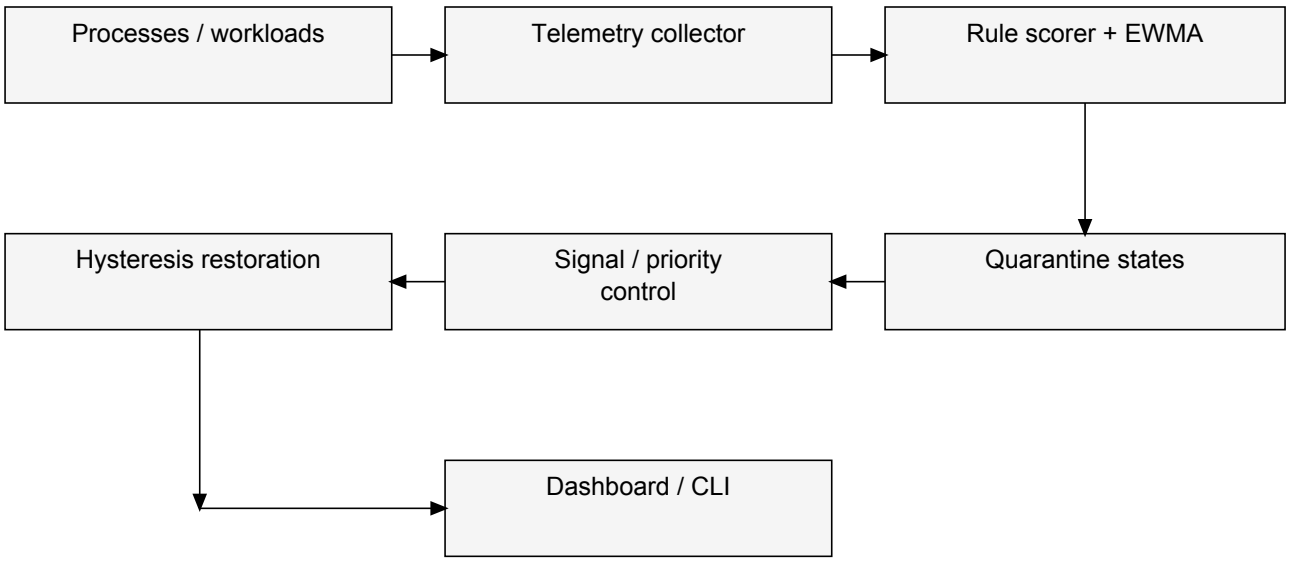



*Figure 1. Existing ARQ observation and control paths.*

| **Component** | **Implementation responsibility** |
|---|---|
| ProcessMonitor | Collect telemetry through native macOS/Linux interfaces. |
| AnomalyScorer | Maintain EWMA tracking and compute the rule score. |
| RestorationEngine | Arbitrate escalation, hysteresis and optional persistent-offender termination. |
| QuarantineController | Apply priority changes and manage duty-cycle pulser threads. |
| Isolation Forest | Provide an optional shadow signal when an artifact is available. |
| Dashboard / CLI | Expose state and run supported demonstrations and evaluations. |

## 9. Workflow and Graduated States

| State | Action and boundary |
|---|---|
| Normal | Monitor without quarantine restrictions. |
| Watchlist | Record elevated behavior; no CPU-duty restriction. |
| Soft quarantine | Request nice +10 and 40% runnable duty. |
| Hard quarantine | Request nice +19 and 15% runnable duty. |
| Terminated | When configured conditions persist, attempt SIGTERM and then SIGKILL. |

Priority operations depend on permissions. Returning from a larger nice value to a lower value may fail for an unprivileged process; the current code does not turn that failure into a guaranteed priority restoration. Signal pulsing pauses and resumes execution but does not erase application memory or prove that external resources remain consistent.

### 9.1 Original recovery

The legacy engine steps down after repeated low scores. This avoids reacting to one sample but does not distinguish suppressed usage from reduced demand. Termination remains a possible policy action, so a general claim that ARQ never kills processes is incorrect.

### 9.2 Proposed research path

The new module is opt-in and operates on one already quarantined process. It tests one larger duty allocation with restricted-release-restricted windows. It does not replace the default dashboard restoration path.

## 10. Implementation Details

| Source area | Role |
|---|---|
| arq/core/monitor.py | Native telemetry; current macOS tests crash on this host. |
| arq/core/anomaly_scorer.py | Rule-based composite score and profile updates. |
| arq/core/quarantine_controller.py | Signal pulser, priority changes and cleanup. |
| arq/core/restoration_engine.py | Existing score-driven stage transitions. |
| arq/core/ml_pipeline.py / ml_training.py | Feature extraction, optional model interfaces, training and calibration. |
| arq/web/server.py | ThreadingHTTPServer dashboard backend; not Flask. |
| arq/core/recovery_probe.py | New window contract, policy gates, trace and throttler adapter. |
| research/recovery_probe/evaluate.py | Deterministic synthetic comparison with the legacy policy. |

The earlier continued report called the dashboard Flask-based. The inspected repository uses Python ThreadingHTTPServer; this combined report corrects that description. Isolation Forest is an optional shadow component. A sequence-model interface exists, but no trained LSTM deployment or LSTM accuracy is established.

### 10.1 Bracketed Recovery Verification

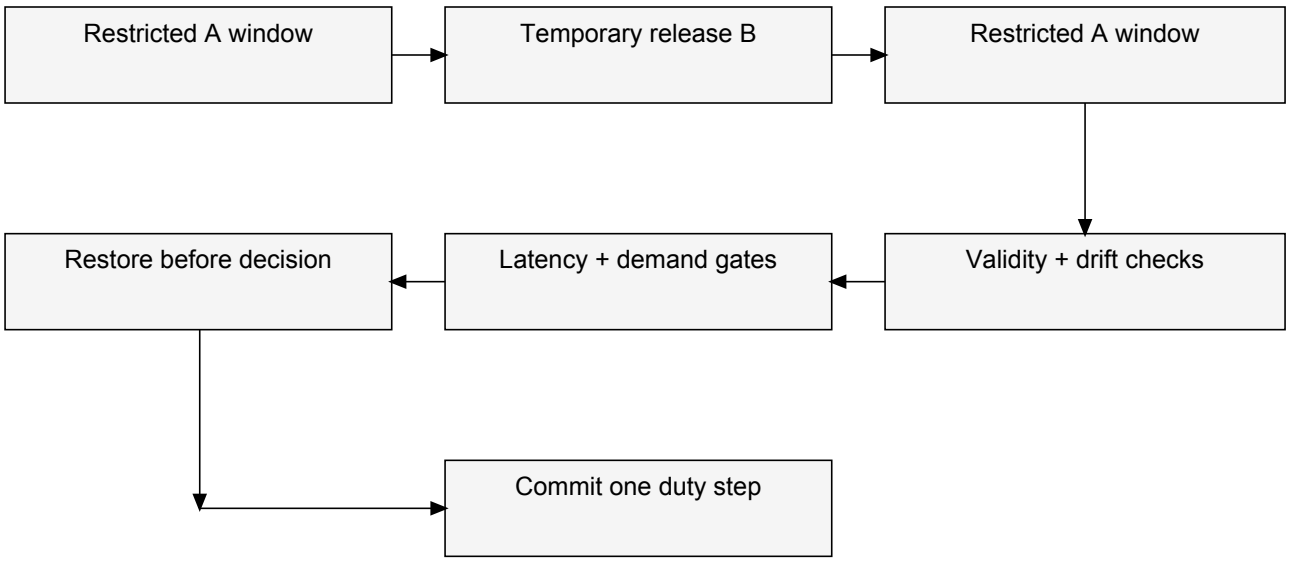



*Figure 2. Opt-in bracketed verification before one duty relaxation.*

A is the current restricted duty allocation. B is one proposed relaxation. Each B measurement is surrounded by A measurements so that changing control conditions can make the experiment inconclusive rather than trigger restoration. The runner reapplies A in a finally block before a positive decision is committed.

The policy checks process incarnation, finite healthy observations, sample count, reported duty and duration, offered-load stability, control latency drift, absolute and relative protected-service p95, and CPU seconds normalized by the B grant. An adapter changes an existing ARQ throttler without changing its stage or niceness.

### 10.2 Default Research Policy

| Parameter | Default |
|---|---|
| Control / proposed duty in evaluation | 0.15 / 0.40 |
| Window length / complete rounds | 0.5 seconds / 2 A-B-A rounds |
| Nominal additional grant budget | 0.30 one-core seconds per call |
| Protected-service p95 SLO | 20 ms |
| Permitted release-induced p95 increase | 2 ms |
| Permitted A-window p95 drift | 1 ms |
| Permitted offered-load drift | 10% |
| Maximum B grant-normalized CPU consumption | 0.70 |
| Minimum latency observations per window | 20 |

The budget reserves (B - A) x window length x rounds: 0.25 nominal additional core-seconds for the evaluation settings. This is a reservation for requested allocation, not a real-time execution guarantee. A delayed callback may overrun a window before the runner can reject its measurement.

### 10.3 Required live adapter behavior

Live use needs process-creation identity checks, measured effective actuation, bounded sampling, a session lock excluding other controllers, an external watchdog and a cumulative repeated-probe budget. Those deployment components are not implemented by the new module. No live performance result is claimed for it.

## 10.4 Supplied Dashboard Output

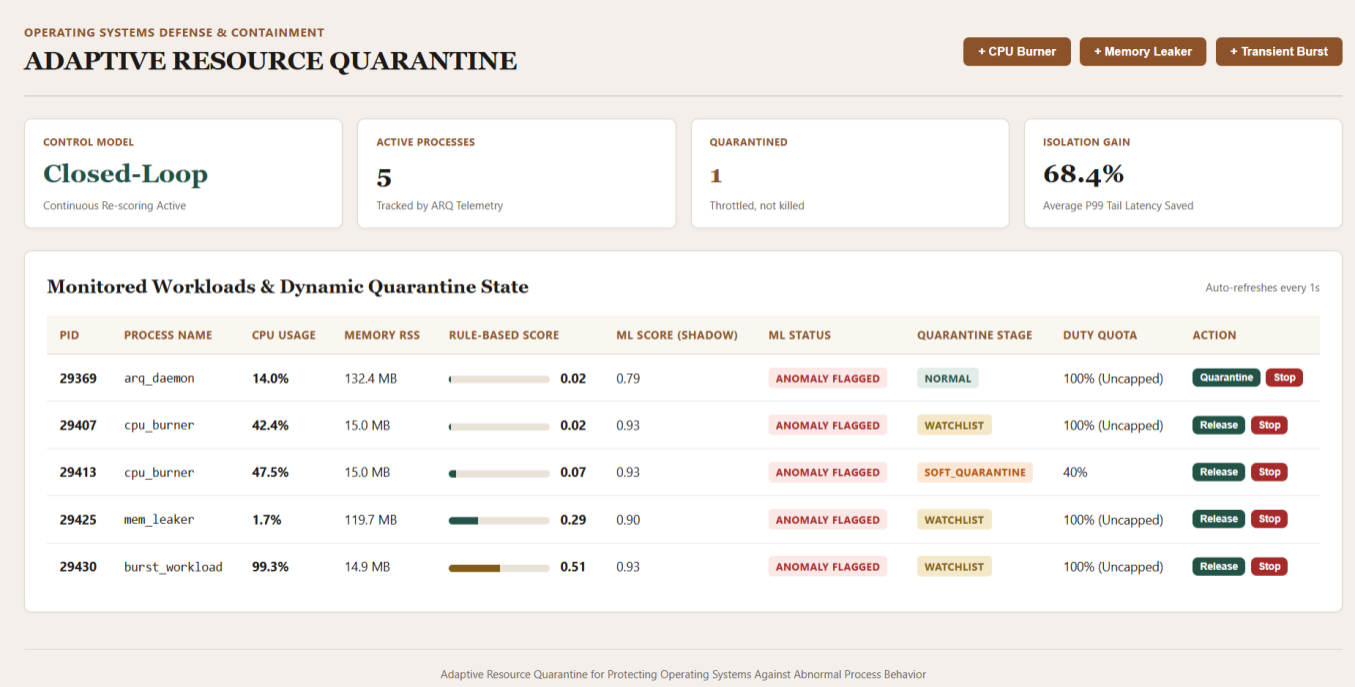



Figure 3. Historical dashboard screenshot supplied in the earlier ARQ Review II report. It is included as the project interface record, not a fresh run of the recovery-probe extension.

The interface presents process resource use, rule scores, shadow-mode ML output and quarantine states. It provides a demonstration surface; the interface itself does not validate resource isolation, restoration correctness or anomaly accuracy.

## 10.5 Reproducible commands

```
python -m unittest tests.test_recovery_probe -v
PYTHONPATH=. python research/recovery_probe/evaluate.py
python -m arq.cli demo
python -m arq.cli benchmark --duration 6.0 --markdown
```

The first two commands reproduce the new tests and synthetic evidence. The last two belong to the existing live prototype and require a validated telemetry environment. Do not treat the historical screenshot as proof that they passed on the current host.

## 11. Current Functional Output

```
test_absolute_slo_rejects
  (tests.test_recovery_probe.TestRecoveryProbe.test_absolute_slo_rejects) ... ok
test_adapter_updates_actual_throttler
  (tests.test_recovery_probe.TestRecoveryProbe.test_adapter_updates_actual_throttler) ... ok
test_budget_blocks_actuation
  (tests.test_recovery_probe.TestRecoveryProbe.test_budget_blocks_actuation) ... ok
test_commit_failure_restores
  (tests.test_recovery_probe.TestRecoveryProbe.test_commit_failure_restores) ... ok
test_drift_is_inconclusive
  (tests.test_recovery_probe.TestRecoveryProbe.test_drift_is_inconclusive) ... ok
test_invalid_observations_fail_closed
  (tests.test_recovery_probe.TestRecoveryProbe.test_invalid_observations_fail_closed) ... ok
test_invalid_policy (tests.test_recovery_probe.TestRecoveryProbe.test_invalid_policy) ... ok
test_load_drift_is_inconclusive
  (tests.test_recovery_probe.TestRecoveryProbe.test_load_drift_is_inconclusive) ... ok
test_protected_harm_rejects_even_below_absolute_slo (tests.test_recovery_probe.TestRecoveryProbe
  .test_protected_harm_rejects_even_below_absolute_slo) ... ok
test_recovered_demand_commits_after_two_complete_brackets (tests.test_recovery_probe.TestRecover
  yProbe.test_recovered_demand_commits_after_two_complete_brackets) ... ok
test_sampler_exception_restores
  (tests.test_recovery_probe.TestRecoveryProbe.test_sampler_exception_restores) ... ok
test_throttled_cpu_is_not_recovery
  (tests.test_recovery_probe.TestRecoveryProbe.test_throttled_cpu_is_not_recovery) ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.002s

OK
```

The log above is the retained execution output for the new module. Tests use simulated observations and controlled/fake actuators. They do not send signals to user workloads or validate native telemetry.

### 11.1 Example Decision Trace

| Observation | Decision implication |
|---|---|
| A: duty 0.15, CPU consumption 0.15 cores | Low absolute usage alone is compatible with a fully saturated restriction. |
| B: duty 0.40, CPU consumption 0.40 cores | Normalized consumption = 1.0; exceeds the 0.70 recovery gate. |
| Second A: duty 0.15 | Restriction is reinstated before the decision is evaluated. |
| Stable load and protected latency | Necessary checks, but insufficient when CPU demand remains masked. |
| Result: restriction_masked_demand | Retain A. Do not mark the process as recovered. |

This is a worked example of the explicit synthetic model, not an operating-system trace. The raw evaluation file retains all 500 seeded trials, reasons and final duties. If normalized-demand checking is removed, the hidden-demand cases pass in this model, showing why the gate matters under these assumptions.

### 11.2 Implementation status

Existing monitoring/control and the new verification runner are separate paths. The extension has a tested bridge to an active throttler, but it is not enabled in the web dashboard. The combined project therefore remains a prototype with an experimental policy extension.

# ADAPTIVE RESOURCE QUARANTINE

*Performance evaluation, comparisons, conclusions and future work*

**REVIEW III**

Rithika.S (24BCE2984)

K. Vaishnavi (24BCT0183)

Vellore Institute of Technology

Operating Systems Lab | BCSE303P

Slot: L15 + L16

Faculty: Prof. Anisha M. Lal

Combined review record and technical disclosure
30 October 2026

Project repository

## 12. Evaluation Method and Functional Results

The current evaluation uses isolated module test processes, retained logs and deterministic constructed scenarios. Results are separated from historical measurements recorded in the repository README. The checked baseline is commit 40e3d28; the extension was developed locally with AI assistance.

| Test module | Current outcome |
|---|---|
| test_anomaly_scorer | 5 passing test methods |
| test_ml_pipeline | 7 passing test methods |
| test_quarantine | 5 passing test methods |
| test_restoration | 3 passing test methods |
| test_recovery_probe | 12 passing test methods |
| ML training | Import failed: optional joblib unavailable. |
| Native monitor | Process exited with signal 11 on macOS. |

Thirty-two test methods passed across the five passing modules, including twelve new recovery-probe tests. The complete suite does not pass on this host. The unchanged native monitor crash blocks a credible live evaluation until investigated; no test count is used to imply full platform validation.

### 12.1 What the new tests exercise

Tests cover recovered and still-demanding processes, absolute and relative latency rejection, offered-load/control drift, insufficient or non-finite observations, identity/actuation mismatch, budget rejection before actuation, sampler exceptions, failed final commits and the existing-throttler bridge.

## 13. Reported Existing Benchmark

The current repository README reports one six-second benchmark run. These are historical reported values, not measurements rerun in this task. The baseline, static termination and ARQ cases have mixed latency outcomes.

| Strategy | P95 ms | P99 ms | Requests/s |
|---|---|---|---|
| Unprotected | 0.16 | 0.43 | 49.2 |
| Static kill | 0.29 | 0.35 | 48.9 |
| ARQ | 0.26 | 0.37 | 48.9 |

ARQ has lower reported P99 than the unprotected run but higher P95; static kill has a lower P99 than ARQ. One short run does not establish a consistent advantage or statistical significance. It is not a benchmark of the new probe policy.

### 13.1 State-loss and completion limitations

The benchmark runner assigns its state-loss percentages and completion flags directly in code. It does not compare application checkpoints, output hashes, durable writes or completed work. Consequently, the reported 0% state-loss value cannot be presented as a measured integrity guarantee.

### 13.2 ML evidence boundary

The README reports normal-telemetry flag rates of 100% for an original model and 2.6% for a candidate, alongside missed synthetic stress scenarios. The underlying dataset and trained artifacts are not present in this checkout. These reported values are not independently reproduced here and do not establish precision, recall or F1. Shadow ML does not decide enforcement.

## 14. Synthetic Recovery-Policy Evaluation

The deterministic model sets observed CPU to min(demand, duty). A recovered workload has demand sampled between 0.08 and 0.20 cores; a still-demanding workload is sampled between 0.80 and 1.00. Protected latency is constructed with small seeded noise. Repeated within-window latency values are not independent request measurements.

| Constructed scenario | Trials | Legacy relaxes | Probe relaxes |
|---|---|---|---|
| recovered | 100 | 100 | 100 |
| masked_demand | 100 | 100 | 0 |
| protected_harm | 100 | 100 | 0 |
| control_drift | 100 | 100 | 0 |
| load_change | 100 | 100 | 0 |

The legacy policy receives its ordinary CPU telemetry. The probe receives additional controlled-release and protected-service observations. This compares complete policies with different inputs, not equal-information algorithms. Drift and load-change cases are treated as inconclusive; rejecting them is not proof that their processes are faulty.

### 14.1 Ablation

Removing the normalized-demand check permits relaxation in 100 of 100 hidden-demand cases in this constructed model. The ablation supports the role of that gate under the model, but it does not demonstrate field accuracy or an independently discovered physical effect.

### 14.2 Reproducibility

Results and raw trials are retained in research/recovery_probe/results/synthetic-comparison.json. The simulation uses seeds 0-99 for each scenario. No invented live latency, CPU savings or recovery-time figures are added.

## 15. Comparison with Existing Systems

| Approach | Established behavior | ARQ boundary |
|---|---|---|
| cpulimit | Signal-based CPU restriction. | No novelty claim for SIGSTOP/SIGCONT. |
| Linux cgroup v2 | Kernel CPU/memory/I/O resource controls. | ARQ signal pulsing is not equivalent to a cgroup sandbox. |
| CPI2 [10] | Detect interference and throttle likely offenders. | Adaptive throttling of suspected offenders is known. |
| Heracles / PARTIES [6,8] | Use feedback and QoS observations to adjust allocations. | Protected-latency control alone is not a new contribution. |
| Caladan / Shenango [7,9] | Fast core-allocation mechanisms for services. | No comparable microsecond scheduler or deployment evidence. |
| New ARQ probe | A-B-A verification with demand and drift gates. | Synthetic policy evidence only; novelty unresolved. |

Published systems were not executed in this evaluation. This is a mechanism comparison, not a claim that ARQ outperforms them. The specific conjunction of bracketed measurements and acceptance gates needs a full prior-art and inventive-step review.

## 16. Project Outcome and Team Record

The deliverable comprises the ARQ source, a local opt-in recovery-verification module, twelve added regression tests, a five-scenario synthetic evaluation, logs and a technical disclosure. It supports a bounded research/prototype outcome rather than a finished production resource manager.

| Person / work area | Record |
|---|---|
| Rithika.S (24BCE2984) | Team identity taken from supplied Review I. Individual module attribution requires team confirmation. |
| K. Vaishnavi (24BCT0183) | Team identity taken from supplied Review I. Individual module attribution requires team confirmation. |
| New research extension | Code, tests and drafting developed with AI assistance during this chat; human inventorship is not inferred. |

## 17. Conclusion and Future Work

ARQ makes process observation and graduated control inspectable. Its original recovery rule has an important limitation: low readings may be caused by the restriction itself. The new opt-in experiment checks a proposed allocation increase with additional measurements and restores the original setting before deciding. Tests establish the implemented behavior on defined inputs; simulation illustrates why demand and stability gates can matter.

Next work is to repair and validate native telemetry, measure actual actuation, add a watchdog and cumulative probing budget, integrate the policy under a shared control lock and run repeated live trials with labelled workloads. Evaluate multicore demand, memory/I/O pressure, application progress and data integrity. Patent review must assess the exact claim combination and the public-disclosure timeline; a patent grant or filing is not claimed.

## 18. References

[1] Ibidunmoye, Hernandez-Rodriguez and Elmroth (2015). Performance Anomaly Detection and Bottleneck Identification. ACM Computing Surveys.

https://doi.org/10.1145/2791120

[2] Liu, Ting and Zhou (2008). Isolation Forest. ICDM.

https://www.lamda.nju.edu.cn/publication/icdm08b.pdf

[3] Hochreiter and Schmidhuber (1997). Long Short-Term Memory. Neural Computation.

https://doi.org/10.1162/neco.1997.9.8.1735

[4] Malhotra et al. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection.

https://arxiv.org/abs/1607.00148

[5] Somayaji (2002). Operating System Stability and Security through Process Homeostasis. Doctoral thesis.

https://www.cs.unm.edu/~forrest/dissertations/smoayaji-dissertation.pdf

## 18. References (continued)

[6] Lo et al. (2016). Improving Resource Efficiency at Scale with Heracles. ACM TOCS.

https://research.google/pubs/improving-resource-efficiency-at-scale-with-heracles/

[7] Fried et al. (2020). Caladan: Mitigating Interference at Microsecond Timescales. OSDI.

https://www.usenix.org/conference/osdi20/presentation/fried

[8] Chen, Delimitrou and Martinez (2019). PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. ASPLOS.

https://www.csl.cornell.edu/~martinez/doc/asplos19.pdf

[9] Ousterhout et al. (2019). Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. NSDI.

https://www.usenix.org/conference/nsdi19/presentation/ousterhout

[10] Zhang et al. (2013). CPI2: CPU performance isolation for shared compute clusters. EuroSys.

https://research.google/pubs/cpi2-cpu-performance-isolation-for-shared-compute-clusters/

## 18. References (continued)

[11] Delimitrou and Kozyrakis (2014). Quasar: Resource-Efficient and QoS-Aware Cluster Management. ASPLOS.

https://www.csl.cornell.edu/~delimitrou/papers/2014.asplos.quasar.pdf

[12] Delimitrou and Kozyrakis (2013). Paragon: QoS-aware scheduling for heterogeneous datacenters. ASPLOS.

https://mast.stanford.edu/pubs/paragon_qos_aware_scheduling_for_heterogeneous_datacenters/

[13] Mars et al. (2011). Bubble-Up: Increasing Utilization In Modern Warehouse Scale Computers Via Sensible Co-Locations. MICRO.

https://research.google/pubs/bubble-up-increasing-utilization-in-modern-warehouse-scale-computers-via-sensible-co-locations/

[14] Banga, Druschel and Mogul (1999). Resource Containers: A New Facility for Resource Management in Server Systems. OSDI.

https://www.usenix.org/conference/osdi-99/presentation/resource-containers-new-facility-resource-management-server-systems

[15] Hindman et al. (2011). Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. NSDI.

https://www.usenix.org/conference/nsdi11/mesos-platform-fine-grained-resource-sharing-data-center

## 18. References (continued)

[16] Verma et al. (2015). Large-scale cluster management at Google with Borg. EuroSys.

https://research.google.com/pubs/archive/43438.pdf

[17] Schwarzkopf et al. (2013). Omega: flexible, scalable schedulers for large compute clusters. EuroSys.

https://research.google/pubs/omega-flexible-scalable-schedulers-for-large-compute-clusters/

[18] Ghodsi et al. (2011). Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. NSDI.

https://www.usenix.org/conference/nsdi11/dominant-resource-fairness-fair-allocation-multiple-resource-types

[19] Rzadca et al. (2020). Autopilot: Workload Autoscaling at Google Scale. EuroSys.

https://research.google/pubs/autopilot-workload-autoscaling-at-google-scale/

[20] Kulkarni et al. (2019). Pliant: Leveraging Approximation to Improve Datacenter Resource Efficiency. HPCA.

https://www.csl.cornell.edu/~nsk49/publication/pliant_hpca/

### 18.1 Technical and Patent Sources


ARQ source repository

https://github.com/vaishnavikarri08-crypto/adaptive-resource-quarantine

cpulimit

https://github.com/opsengine/cpulimit

Linux cgroup v2 documentation

https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html

US7530072B1 - suspicious-thread quarantine

https://patents.google.com/patent/US7530072B1/en

US9280662B2 - isolation of misbehaving processes

https://patents.google.com/patent/US9280662B2/en

US20230128059A1 - dynamic allocation for tail-latency SLOs

https://patents.google.com/patent/US20230128059A1/en

Indian Patents Act Section 3

https://www.ipindia.gov.in/acts/patent-act-1970/section-3

Official CRI guideline collection

https://ipindia.gov.in/resource/patents-resources-guidelines


Source documents supplied by the user: ARQ Review I Final Report and ARQ Review II Continued Report Arial12. Their project information and historical dashboard image were used. The supplied merged OptiGuard report provided the layout and report-plus-IDF structure. Unverified bibliographic entries from the older ARQ report were not carried forward.

# ADAPTIVE RESOURCE QUARANTINE

*Candidate technical disclosure for institutional and patent-agent review*

**INVENTION DISCLOSURE FORMAT - IDF B**

Rithika.S (24BCE2984)

K. Vaishnavi (24BCT0183)

Vellore Institute of Technology

Operating Systems Lab | BCSE303P

Slot: L15 + L16

Faculty: Prof. Anisha M. Lal

Combined review record and technical disclosure
30 October 2026

Project repository

Draft technical content following the supplied IDF-B section order. Institutional document control, applicant details and inventor declarations must be confirmed by VIT. This is not a filed patent application or a grant.

## 1. Title of the invention

Bracketed Verification of CPU-Quarantine Relaxation Using Grant-Normalized Consumption and Protected-Service Stability Checks

## 2. Field / Area of invention

Computer science and engineering; operating-system resource control, process scheduling, performance isolation and monitored restoration of restricted process execution.

## Disclosure scope

The candidate is the new opt-in recovery-verification protocol, not the broad idea of adaptive quarantine. It is implemented as a serialized callable experiment with a bridge to an existing ARQ throttler. It is not enabled in default dashboard enforcement.

## Inventor / applicant record

The project team consists of Rithika.S (24BCE2984) and K. Vaishnavi (24BCT0183), as supplied in the course report. This does not establish legal inventorship. Human conception, ownership, applicant details, institutional policy and the role of AI assistance need separate confirmation.

### 3. Prior Patents and Publications

| Reference | Established overlap | Candidate boundary |
|---|---|---|
| US7530072B1 (2009) | Quarantine suspicious threads, reduce CPU and priority, restore after exit criteria. | Broad quarantine-and-recovery claims overlap strongly. |
| US9280662B2 (2016) | Keep misbehaving processes operating in isolation and perform remediation. | Candidate tests relaxation of an existing restriction rather than migration. |
| Heracles [6]; PARTIES [8] | Feedback allocation using latency or QoS observations. | Latency-aware relaxation alone is insufficient as a distinction. |
| CPI2 [10]; Bubble-Up [13] | Interference diagnosis, throttling and active resource perturbation. | Active experimentation and selective control are known. |
| US20230128059A1 (2023) | Dynamic resources for latency-sensitive application SLOs. | Full claims and patent-family comparison remain necessary. |
| cpulimit | Duty-cycle CPU control using stop/continue signals. | The actuator is not claimed as new. |

This comparison is preliminary. The exact combination must be checked against full disclosures, patent families and non-patent literature. No search result establishes novelty merely because its title differs. Expired patents may still constitute prior art.

## 4. Summary of the Invention

A resource controller retains a restricted allocation A for a process whose observed usage may be low because of that restriction. Before relaxing it to B, the controller reserves a configured nominal additional grant and performs repeated A-B-A observation groups. It records CPU consumption, the effective allocation, protected-service latency and offered load for each group. The restriction is reapplied before the decision is evaluated.

The proposal is rejected or treated as inconclusive when observations are invalid, the process identity changes, offered load varies too much, restricted-control latency drifts, protected latency exceeds absolute or relative bounds, or CPU consumption substantially saturates the larger grant. A successful decision permits only the tested allocation step. It does not certify full recovery or application correctness.

### Technical problem

A score-only recovery rule can mistake the controller-induced reduction in observed CPU use for reduced underlying demand. This can trigger premature relaxation. The proposed experiment supplies additional intervention and protected-workload observations rather than relying on repeated low restricted scores alone.

### Technical effect to investigate

More selective relaxation of CPU restrictions under an explicit measurement contract. Current tests and synthetic evidence show the intended control behavior; live resource savings, response-time improvement, safety and generalization have not been established.

## 5. Objectives of the Invention

Verify one proposed CPU-duty relaxation through repeated restricted-release-restricted observations.

Check whether the candidate grant remains saturated, rather than interpreting low restricted CPU as recovery.

Evaluate the protected workload against absolute latency and relative-degradation gates.

Reject comparisons affected by offered-load change or restricted-control drift.

Reserve a nominal incremental grant before the experiment, and retain the original restriction on rejection or invalid data.

Keep phase observations and reasons in an auditable decision trace.

### Scope boundary

The implementation addresses one-core normalized CPU consumption and one protected-service observation contract. It does not solve general resource attribution, multicore normalization, memory isolation or application-state restoration. The thresholds are configurable engineering bounds, not derived optimal settings.

## 6. Components and Working Principle

| Component | Function |
|---|---|
| Window | Immutable observation contract: process identity, duty, duration, CPU time, latency samples, rate and health. |
| ProbePolicy | Configurable measurement, demand, latency, drift and budget limits. |
| RecoveryProbe | Run bracketed experiment; validate observations; restore before deciding; commit one step only on success. |
| ProbeResult | Decision, reason, final duty, nominal reservation and phase trace. |
| ExistingThrottlerAdapter | Update an already quarantined ARQ duty-cycle pulser and profile allocation. |
| Caller-supplied sampler / serialization | Provide trustworthy measurements and prevent concurrent control writers; live components remain external. |

### Working principle

Check the reservation before actuation. For each required round: apply A and observe; apply B and observe; return to A and observe. Validate the three-window comparison. Always reapply A on exit from the experiment. If all rounds pass, apply B as the final allocation; otherwise retain A. Callback or rollback failures propagate as exceptions rather than success results.

## 7. Description of the Invention in Detail

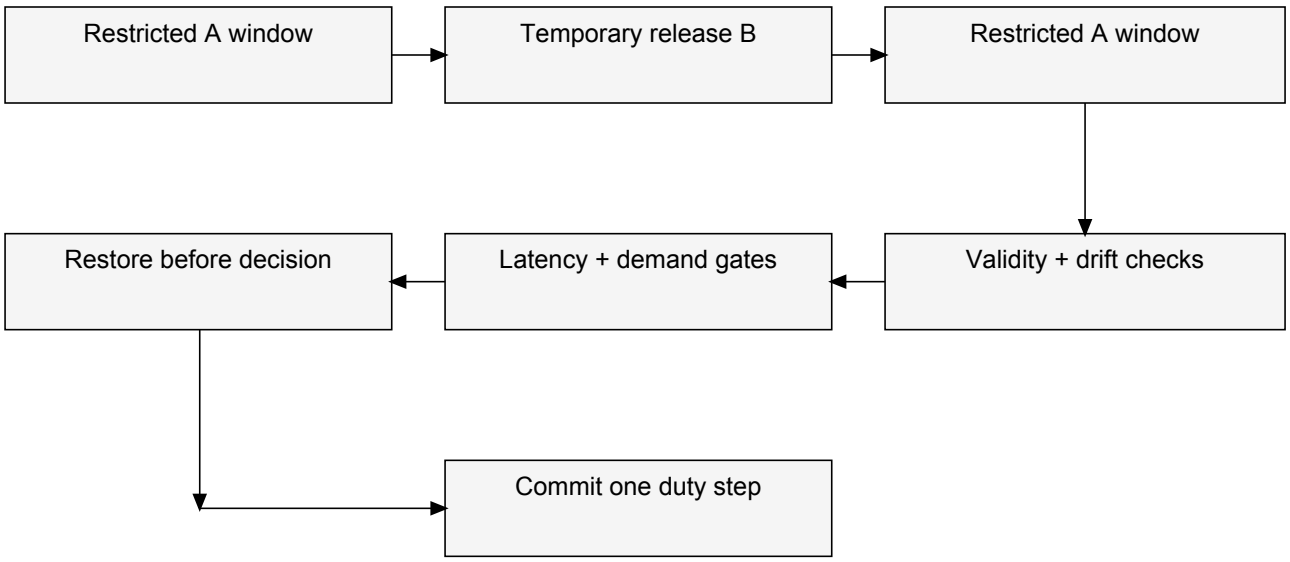



*Figure 2. Opt-in bracketed verification before one duty relaxation.*

### 7.1 Observation and validity contract

The process identity must identify an incarnation, such as PID plus creation identity. Each window must provide finite healthy metrics, sufficient protected requests, a positive offered rate, nonnegative CPU/latency values and agreement with the requested duration and duty. The prototype trusts the callback to obtain these measurements; a nominal requested ratio alone is not verified actual actuation.

### 7.2 Control and demand gates

Control-window p95 drift must stay within 1 ms, offered-rate variation within 10%, and all p95 values within the 20 ms SLO. The probe p95 increase over the median A-window p95 must remain within 2 ms. Probe CPU seconds divided by duration times duty must remain at most 0.70. All values are the default research policy and can be changed.

### 7.3 Algorithm

```
reserve = (B - A) * window_seconds * rounds
if reserve > budget: return budget_exceeded
try:
    for each required round:
        before = apply_and_observe(A)
        trial  = apply_and_observe(B)
        after  = apply_and_observe(A)
        validate identity, samples, duty, duration, health
        check absolute SLO for each window
        reject changed offered load or A-window drift
        reject excessive B latency increase
        reject saturated grant-normalized B consumption
finally:
    apply(A)
if every complete round passed:
    try: apply(B)
    on failure: apply(A); raise
    return verified_step
return retained_restriction_with_reason
```

### 7.4 Limits of the experiment

A-B-A bracketing does not prove causality in the presence of unobserved changes or delayed effects. The method cannot prove that a blocked or externally contended process has low latent demand. It does not bound an unresponsive callback, survive daemon death or guarantee signal delivery. Production use needs a watchdog, process-identity validation, synchronization, repeated-probe limits and actual actuation accounting.

### 7.5 Integration boundary

The new module does not run concurrently with the legacy restoration engine by design contract. The caller must serialize both paths. A positive result changes duty only; stage and niceness remain unchanged. No normal-state declaration, memory restoration or ML-based approval is performed.

## 8. Experimental Validation Results

| Evidence | Result and interpretation |
|---|---|
| New regression methods | 12 passed; simulated observations and fake/controlled actuators. |
| Other passing modules | 20 methods across scorer, inference interface, quarantine and restoration. |
| Broader validation | ML training dependency unavailable; native macOS monitor process crashed. |
| Constructed recovered cases | 100/100 probe approvals under the stated model. |
| Constructed masked-demand cases | 0/100 probe approvals; legacy relaxed in 100/100. |
| Constructed protected harm / drift / load change | 0/100 approvals in each scenario; invalid comparisons are inconclusive. |
| Demand-gate ablation | 100/100 masked-demand approvals when that gate is removed. |

The five modeled groups contain 500 seeded trials. Their outcomes exercise programmed gates, not a representative labelled detection dataset. They cannot establish real-world false-positive rates or improved tail latency. Thirty-two passing methods do not make the entire suite successful.

### Disclosure and publication record

The base repository was publicly accessible when reviewed. Its commit timestamps do not establish first public availability. This report and the new extension are being prepared for repository publication at the user request on 30 October 2026; publication is a disclosure event to record if the push succeeds. Any earlier filing, priority date or applicable exception must be assessed by an Indian patent agent. Making a repository private later would not erase earlier disclosure.

## 9. Aspects Proposed for Protection

The following is a candidate claim skeleton for review, not an examined claim set. Resource quarantine, throttling, feedback control, intervention experiments and defensive cleanup each have prior art. Novelty and inventive step of the proposed conjunction remain unresolved.

1. A computer-implemented method for verifying a proposed relaxation of an existing CPU execution restriction for an identified process incarnation, comprising reserving a nominal additional execution allowance; collecting restricted, relaxed and restricted observation windows; reapplying the restricted allocation before evaluating those windows; evaluating observation validity, offered-load stability, restricted-control latency stability, protected-service latency and relaxed-grant-normalized CPU consumption; and committing the proposed allocation only after the required complete window groups satisfy the configured conditions.

2. The method of candidate claim 1, wherein grant-normalized consumption is CPU seconds consumed in a relaxed window divided by the product of its duration and its reported effective duty allocation.

3. The method of candidate claim 1, wherein protected latency is checked against both an absolute SLO and a permitted increase over the median of the two restricted-control window p95 values.

4. The method of candidate claim 1, wherein changed offered load, changed process incarnation, insufficient observations or control-window drift prevents a positive restoration decision.

5. The method of candidate claim 1, wherein the restricted allocation is reapplied on an exception and before any final successful relaxation, and an unsuccessful final actuation triggers another restoration attempt.

6. A processor and memory configured to execute the method of candidate claim 1 while retaining a trace of the observation phases, measurements and decision reason.

## 10. Technology Readiness Level

| Level | Definition | Selection |
|---|---|---|
| TRL 1 | Basic principles observed. | |
| TRL 2 | Technology concept formulated. | |
| TRL 3 | Experimental proof of concept. | Selected |
| TRL 4 | Technology validated in a laboratory. | |
| TRL 5 | Technology validated in a relevant environment. | |
| TRL 6 | Technology demonstrated in a relevant environment. | |
| TRL 7 | System prototype demonstrated in an operational environment. | |
| TRL 8 | System complete and qualified. | |
| TRL 9 | Actual system proven in an operational environment. | |

TRL 3 is the proposed level for the new recovery-verification mechanism: implemented logic, regression tests and an explicit synthetic experiment. Validated live sampling and production deployment are not established.

### Institutional review items

Confirm inventor/applicant details and actual human contributions; inspect full prior-art disclosures; assess Indian eligibility and inventive step; establish all publication and filing dates; resolve native telemetry and measurement issues before making live technical-effect claims. The report does not certify plagiarism or AI-content percentages.